



Materia: SEMINARIO FINAL DE INGENIERIA EN SOFTWARE

Profesor: JORGE HUMBERTO CASSI

Alumno:

Victor Emmanuel Brizzi

Córdoba, 26/04/2026

INDICE

INDICE	1
Titulo <i>Modelo Automatizado De Evaluación Continua De Requerimientos No Funcionales</i> ...	2
Introducción.....	2
Antecedentes	2
Descripción del área problemática.....	3
Justificación	4
Objetivo General Del Proyecto.	5
Objetivos específicos del proyecto	5
Marco Teórico Referencial	5
1. Dominio del problema.....	5
2. Tecnologías de la Información y la Comunicación (TIC)	6
2.1 Herramientas de evaluación	6
2.2 Tecnologías de desarrollo	6
2.3 Generación de reportes.....	7
3. Competencia.....	7
Diseño Metodológico	7
1. Herramientas metodológicas y de desarrollo	7
2. Recolección de datos.....	8
3. Planificación del proyecto	9
Relevamiento.....	10
1. Relevamiento estructural	10
2. Relevamiento funcional.....	11
2.1 Roles identificados	11
2.2 Funciones.....	11
3. Procesos relevados	11
4. Relevamiento de documentación	12
Procesos de negocio.....	13
Evaluación actual de requerimientos no funcionales	13
Referencias.....	13

Título

Modelo Automatizado De Evaluación Continua De Requerimientos No Funcionales

Introducción

El presente trabajo tiene como objetivo desarrollar un framework automatizado para la evaluación de requerimientos no funcionales en aplicaciones de software, mediante la integración de herramientas de análisis y la utilización de métricas objetivas. La propuesta busca generar indicadores cuantificables y reportes estandarizados que permitan obtener una visión integral del estado de calidad de un sistema, facilitando la toma de decisiones dentro del ciclo de desarrollo.

Antecedentes

En el desarrollo de software moderno, la calidad de los sistemas no se limita únicamente al cumplimiento de requerimientos funcionales, sino que también depende de atributos no funcionales como el rendimiento, la seguridad, la usabilidad y la mantenibilidad. Estos atributos han sido formalmente definidos en estándares como ISO/IEC 25010, el cual establece un modelo de calidad que permite evaluar diferentes características de los productos de software.

Sin embargo, los requerimientos no funcionales presentan dificultades particulares en su especificación y validación. Según Sommerville (2011), estos requerimientos suelen ser más difíciles de verificar que los requerimientos funcionales, lo que genera ambigüedad en su interpretación y dificulta su evaluación a lo largo del ciclo de desarrollo.

En la práctica, la evaluación de estos atributos suele realizarse en etapas tardías del desarrollo o incluso en producción, lo que incrementa el riesgo de fallos y retrabajo. En este sentido, Forsgren, Humble y Kim (2018) destacan que la integración de prácticas de calidad dentro del flujo de desarrollo permite mejorar la estabilidad y el desempeño de los sistemas, reduciendo problemas detectados en etapas finales.

Por otra parte, la evaluación de los requerimientos no funcionales suele apoyarse en herramientas especializadas que abordan aspectos específicos de forma aislada, como el rendimiento, la seguridad o la calidad del código. Esta fragmentación dificulta la obtención de una visión integral del estado de calidad de una aplicación y limita la capacidad de tomar decisiones basadas en métricas unificadas.

Asimismo, enfoques modernos como los propuestos en Site Reliability Engineering enfatizan la importancia de la medición continua y el uso de métricas para garantizar la confiabilidad de los sistemas, destacando la necesidad de contar con mecanismos sistemáticos de medición a lo largo del tiempo (Beyer et al., 2016).

En este contexto, se evidencia la necesidad de contar con soluciones que permitan integrar la evaluación de los requerimientos no funcionales dentro del ciclo de desarrollo, de manera automatizada, continua y basada en métricas, superando las limitaciones de los enfoques actuales.

Descripción del área problemática

En el contexto actual del desarrollo de software, los equipos técnicos requieren contar con información confiable y oportuna sobre la calidad de sus aplicaciones más allá del correcto funcionamiento funcional. En particular, resulta fundamental evaluar atributos no funcionales como el rendimiento, la seguridad y la mantenibilidad, debido a su impacto directo en el comportamiento del sistema en entornos productivos.

No obstante, en entornos reales de desarrollo, la evaluación de estos atributos suele integrarse de manera limitada dentro del ciclo de trabajo. En muchos casos, los equipos utilizan herramientas específicas de forma independiente, sin una estrategia común que permita unificar criterios o resultados.

Como consecuencia, la información obtenida a partir de estos análisis se presenta de forma dispersa, dificultando la interpretación global del estado de calidad de una aplicación. Esta situación obliga a los equipos a realizar análisis manuales y a basar sus decisiones en múltiples fuentes de información no integradas.

Adicionalmente, la falta de mecanismos sistemáticos para registrar y comparar resultados a lo largo del tiempo limita la capacidad de identificar tendencias, mejoras o regresiones en la calidad del software.

En este escenario, se evidencia la necesidad de contar con herramientas que permitan centralizar la evaluación de requerimientos no funcionales, facilitando la obtención de indicadores claros, comparables y orientados a la toma de decisiones.

Justificación

El presente proyecto surgió a partir de la necesidad de mejorar la forma en que se evalúan los requerimientos no funcionales en el desarrollo de software. Estos aspectos, como el rendimiento, la seguridad y la usabilidad, suelen analizarse de manera aislada, utilizando herramientas específicas y sin un criterio unificado que permita interpretar los resultados de forma integral. Esta situación dificultaba la comprensión del estado real de calidad de una aplicación y limitaba la capacidad de tomar decisiones basadas en métricas claras.

En este sentido, el proyecto buscó cubrir la necesidad de contar con un enfoque automatizado, continuo y estandarizado para la evaluación de tales requerimientos, permitiendo integrar estos análisis dentro del ciclo de desarrollo. De esta manera, se facilitó la detección temprana de problemas, se redujo el riesgo de fallos en producción y se mejoró la calidad general del software.

Desde el punto de vista tecnológico, la propuesta introdujo un modelo que no solo permitió medir distintos atributos de calidad, sino también interpretarlos mediante un sistema de scoring unificado y analizarlos a lo largo del tiempo. Esto representó un avance respecto de los enfoques tradicionales, que suelen enfocarse en métricas aisladas sin considerar su evolución ni su impacto conjunto.

Asimismo, el valor del proyecto para las organizaciones radicó en la posibilidad de contar con información objetiva y centralizada sobre la calidad de sus sistemas, lo que permitió optimizar procesos de desarrollo, mejorar la toma de decisiones y reducir costos asociados a la detección tardía de errores. En un contexto donde los sistemas informáticos son críticos para la operación de las organizaciones, disponer de este tipo de herramientas resultó altamente relevante.

Finalmente, el proyecto introdujo una innovación a nivel de procesos, al transformar la evaluación de los requerimientos no funcionales en una actividad continua, automatizada y basada en métricas. Esto permitió pasar de un enfoque reactivo, donde los

problemas se detectaban tardíamente, a un enfoque proactivo, donde la calidad fue monitoreada de forma constante a lo largo del ciclo de vida del software.

Objetivo General Del Proyecto.

Desarrollar un framework automatizado para la evaluación de requerimientos no funcionales en aplicaciones de software, basado en la ejecución de pruebas y el uso de métricas objetivas, que permita generar reportes estandarizados con indicadores cuantificables.

Objetivos específicos del proyecto

- I. Definir un conjunto de métricas objetivas para la evaluación de requerimientos no funcionales en las categorías de mantenibilidad, seguridad y rendimiento.
- II. Diseñar un modelo de evaluación que permita integrar, normalizar y analizar los resultados obtenidos a partir de herramientas automatizadas.
- III. Implementar un mecanismo de cálculo de puntajes que permita cuantificar el nivel de cumplimiento de los requerimientos no funcionales evaluados.
- IV. Desarrollar un prototipo funcional que permita ejecutar pruebas automatizadas, procesar los resultados y generar reportes estandarizados.
- V. Validar el funcionamiento del framework mediante la aplicación del prototipo en un caso de estudio.

Marco Teórico Referencial

1. Dominio del problema

En el desarrollo de software moderno, los requerimientos no funcionales (NFR) representan atributos de calidad que determinan cómo un sistema opera, más allá de las funcionalidades que ofrece. Según el estándar ISO/IEC 25010, la calidad del software se compone de características como rendimiento, seguridad y mantenibilidad, entre otras.

Diversos estudios señalan que los requerimientos no funcionales suelen ser difíciles de especificar y medir de forma objetiva, lo que genera ambigüedad en su implementación (Chung et al., 2000). Esta dificultad radica en que estos atributos son transversales y dependen del contexto del sistema.

En la práctica industrial, la evaluación de estos requerimientos se realiza mediante herramientas especializadas que operan de manera aislada. Por ejemplo, la seguridad se evalúa mediante análisis de vulnerabilidades, mientras que el rendimiento se analiza mediante pruebas de carga. Sin embargo, estas evaluaciones carecen de un modelo unificado que permita integrar sus resultados (Bass, Clements & Kazman, 2012).

En este contexto, surge la necesidad de desarrollar un enfoque que permita integrar la evaluación de múltiples atributos de calidad en un modelo cuantificable y estandarizado, facilitando la toma de decisiones sobre el estado de un sistema.

2. Tecnologías de la Información y la Comunicación (TIC)

Para la implementación del framework propuesto se consideran tecnologías y herramientas que permiten la automatización de pruebas, la integración de resultados y la generación de reportes.

2.1 Herramientas de evaluación

Para la evaluación de los requerimientos no funcionales se utilizarán herramientas especializadas.

Para la mantenibilidad se empleará SonarQube, que permite analizar código fuente y obtener métricas de calidad.

Para la seguridad se utilizará OWASP ZAP, que permite detectar vulnerabilidades en aplicaciones web mediante pruebas automatizadas.

Para el rendimiento se utilizará k6, que permite evaluar el comportamiento del sistema bajo carga.

2.2 Tecnologías de desarrollo

El framework será desarrollado utilizando Node.js, debido a su capacidad para manejar procesos asíncronos y su compatibilidad con herramientas de automatización.

La integración entre componentes se realizará mediante APIs REST, utilizando formatos de intercambio de datos como JSON.

Se empleará Docker para la contenedorización del sistema, garantizando la reproducibilidad de las ejecuciones y el aislamiento de los entornos.

Para el almacenamiento de resultados se considerarán soluciones de bases de datos que permitan gestionar el historial de ejecuciones y facilitar el análisis de resultados.

2.3 Generación de reportes

Para la visualización de resultados se utilizará Chart.js, que permite representar gráficamente los indicadores obtenidos.

Asimismo, se utilizará Puppeteer para la generación de reportes en formato PDF a partir de vistas HTML.

La generación de reportes combinará visualizaciones gráficas y exportación en formato PDF, con el objetivo de facilitar tanto el análisis técnico como la comunicación de resultados.

3. Competencia

En la actualidad existen diversas herramientas que permiten evaluar aspectos específicos de la calidad del software de manera independiente.

Por ejemplo, SonarQube se enfoca en la calidad del código, mientras que OWASP ZAP se especializa en seguridad, y k6 en pruebas de rendimiento.

Asimismo, herramientas como Google Lighthouse permiten generar reportes automatizados con puntuaciones, aunque limitadas al análisis de aplicaciones web.

A pesar de la existencia de estas soluciones, no se identifica un enfoque que permita integrar múltiples dimensiones de calidad en un modelo unificado con indicadores cuantificables, lo que justifica el desarrollo del presente proyecto.

Diseño Metodológico

1. Herramientas metodológicas y de desarrollo

Para el desarrollo del presente proyecto se utilizarán diversas tecnologías y herramientas que permitirán implementar el framework propuesto de manera automatizada, escalable y reproducible.

En la capa de backend se empleará Node.js, debido a su capacidad para gestionar operaciones asincrónicas y su compatibilidad con herramientas de automatización, lo cual resulta adecuado para la orquestación de pruebas y procesamiento de resultados.

La comunicación entre los distintos componentes del sistema se realizará mediante APIs REST, permitiendo la integración de herramientas externas y la estandarización del intercambio de datos en formato JSON.

Para la ejecución aislada de pruebas y garantizar la reproducibilidad de los entornos, se utilizará Docker, lo que permitirá encapsular las herramientas de evaluación y evitar dependencias del entorno de ejecución.

En cuanto a la evaluación de requerimientos no funcionales, se integrarán herramientas especializadas como:

SonarQube, para el análisis de mantenibilidad del código.

OWASP ZAP, para la detección de vulnerabilidades de seguridad.

k6, para la ejecución de pruebas de rendimiento.

Para la visualización de resultados se utilizará Chart.js, permitiendo representar gráficamente los indicadores obtenidos.

Asimismo, se empleará Puppeteer para la generación de reportes en formato PDF, facilitando la presentación de resultados en un formato estructurado y profesional.

Estas herramientas permitirán alcanzar el objetivo del proyecto al posibilitar la automatización de pruebas, la integración de resultados y la generación de reportes cuantificables.

El desarrollo del proyecto se llevará a cabo siguiendo un enfoque ágil, basado en el marco de trabajo Scrum, lo que permitirá organizar las actividades en iteraciones incrementales, facilitando la validación continua del prototipo y la adaptación a posibles cambios durante el proceso de desarrollo.

2. Recolección de datos

La recolección de datos se llevará a cabo mediante el análisis de información proveniente de distintas fuentes, tanto teóricas como prácticas, con el objetivo de comprender el dominio del problema y validar el funcionamiento del framework propuesto.

En primer lugar, se realizará una revisión bibliográfica de estándares y modelos de calidad de software, como ISO/IEC 25010, así como de literatura académica relacionada con requerimientos no funcionales y métricas de software.

En segundo lugar, se aplicará una técnica de análisis comparativo (benchmarking) de herramientas existentes, evaluando soluciones actuales del mercado que permiten medir aspectos específicos de la calidad del software, tales como seguridad, rendimiento y mantenibilidad, mediante la definición de criterios de evaluación que permitan identificar sus capacidades, limitaciones y posibilidades de integración.

Finalmente, para la validación del prototipo, se utilizará un caso de estudio, en el cual se ejecutarán las pruebas automatizadas sobre una aplicación de software, permitiendo recolectar datos reales sobre su comportamiento y evaluar los resultados obtenidos.

3. Planificación del proyecto

Para la planificación del desarrollo del proyecto se elaboró un diagrama de Gantt, en el cual se definieron las actividades correspondientes a la primera etapa del trabajo, así como su secuencia y duración estimada.

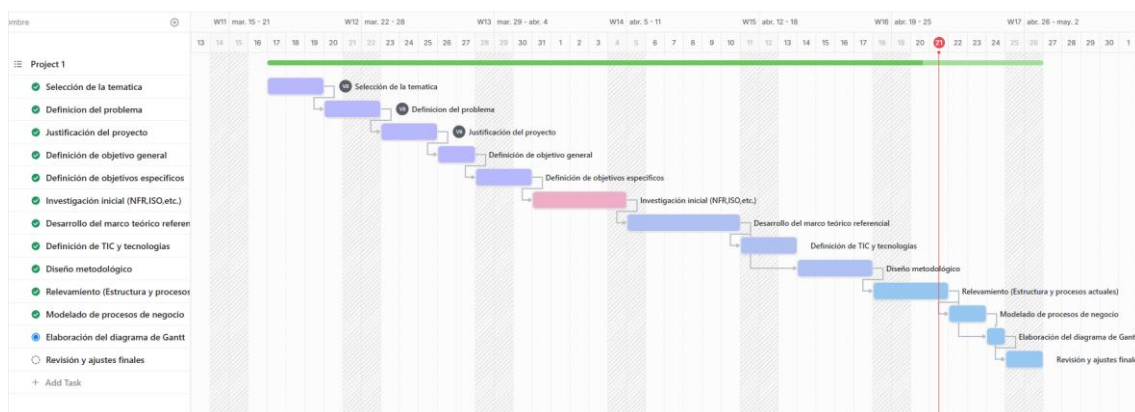
Las actividades contempladas en esta fase incluyen:

- I. Selección de la temática
- II. Definición del problema
- III. Justificación del proyecto
- IV. Definición del objetivo general
- V. Definición de los objetivos específicos
- VI. Investigación inicial sobre requerimientos no funcionales y estándares de calidad
- VII. Desarrollo del marco teórico referencial
- VIII. Definición de tecnologías y herramientas (TIC)
- IX. Diseño metodológico
- X. Relevamiento de la situación actual (estructura y procesos)
- XI. Modelado de procesos de negocio
- XII. Elaboración del diagrama de Gantt
- XIII. Revisión y ajustes finales

La planificación permitió organizar el trabajo de manera estructurada, asegurando el cumplimiento de los plazos establecidos para la presente entrega y la correcta elaboración de los contenidos requeridos.

El plan de actividades se detalla en el siguiente diagrama de Gantt, en el cual se representan las tareas desarrolladas, su secuencia temporal y las relaciones de dependencia entre ellas.

Asimismo, las actividades del proyecto se agrupan en cuatro etapas principales: definición del proyecto, investigación, desarrollo teórico y modelado, permitiendo una organización progresiva desde la conceptualización hasta la representación de los procesos.



Relevamiento

1. Relevamiento estructural

El presente proyecto no se desarrolla en el contexto de una organización real, sino que se basa en una organización modelada, representativa de empresas dedicadas al desarrollo de software.

En este contexto, se considera una empresa de desarrollo que construye aplicaciones web y que requiere evaluar la calidad de sus productos antes de su puesta en producción.

La infraestructura tecnológica típica de este tipo de organizaciones incluye:

- I. Entornos de desarrollo y testing
- II. Repositorios de código fuente

- III. Pipelines de integración continua (CI/CD)
- IV. Aplicaciones desplegadas en entornos de staging o producción

Asimismo, los equipos de trabajo interactúan mediante herramientas de desarrollo colaborativo y utilizan sistemas automatizados para la ejecución de pruebas.

2. Relevamiento funcional

Dado que se trata de una organización modelada, no se define una estructura jerárquica formal, sino que se identifican los roles y funciones involucrados en el proceso de evaluación de calidad del software.

2.1 Roles identificados

- I. Desarrollador: responsable de implementar funcionalidades en el sistema.
- II. QA / Tester: encargado de validar el correcto funcionamiento del sistema mediante pruebas.
- III. DevOps / SRE: responsable de la integración, despliegue y monitoreo del sistema.
- IV. Responsable técnico / arquitecto: encargado de tomar decisiones sobre la calidad y arquitectura del sistema.

2.2 Funciones

- I. Desarrollo de funcionalidades del sistema
- II. Ejecución de pruebas funcionales y no funcionales
- III. Integración continua y despliegue de aplicaciones
- IV. Evaluación de métricas de calidad del software
- V. Toma de decisiones en base a resultados de calidad

3. Procesos relevados

Nombre del proceso: Evaluación de calidad no funcional del software (situación actual)

Roles: Desarrollador, QA, DevOps, Responsable técnico

Pasos:

- I. El desarrollador implementa cambios en el código fuente y los integra en el repositorio.
- II. Se despliega la aplicación en un entorno de testing o staging.
- III. El equipo de QA ejecuta pruebas funcionales sobre la aplicación, enfocándose principalmente en la validación de funcionalidades.
- IV. De manera aislada, algunos miembros del equipo ejecutan herramientas específicas para evaluar aspectos particulares, como análisis de código, pruebas de seguridad o pruebas de rendimiento.
- V. Los resultados obtenidos por estas herramientas no siguen un formato estandarizado y suelen analizarse de manera individual.
- VI. No existe un mecanismo unificado que permita integrar los resultados obtenidos ni generar indicadores cuantificables de calidad no funcional.
- VII. La evaluación final del estado del sistema se realiza de manera subjetiva, basada en la experiencia del equipo técnico y en la interpretación manual de los resultados disponibles.

4. Relevamiento de documentación

En el contexto actual, se identifican los siguientes artefactos generados durante el proceso:

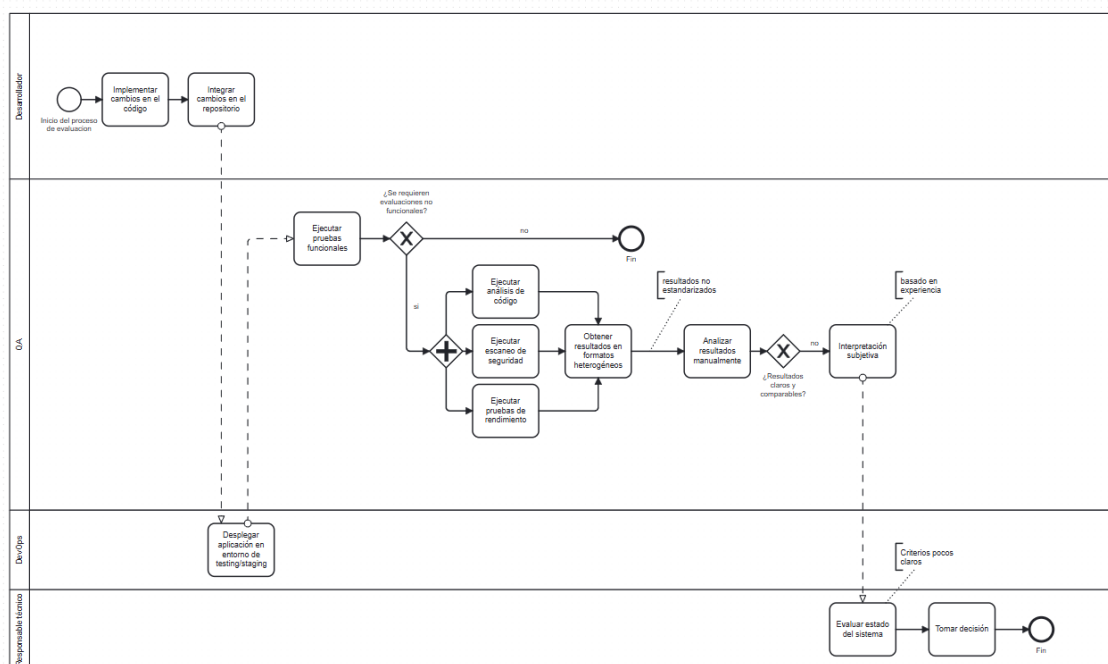
- I. Reportes individuales de herramientas de análisis de código
- II. Resultados de escaneos de seguridad
- III. Resultados de pruebas de rendimiento
- IV. Logs y registros de ejecución de pruebas

Estos documentos se encuentran desacoplados entre sí, presentan distintos formatos y no permiten obtener una visión integral del estado de calidad del sistema.

Procesos de negocio

Evaluación actual de requerimientos no funcionales

El proceso de negocio modelado refleja la situación actual de la evaluación de requerimientos no funcionales en organizaciones de desarrollo de software, evidenciando la ejecución aislada de herramientas, la falta de estandarización de resultados y la dependencia de análisis manual para la toma de decisiones.



Referencias

- Bass, L., Clements, P., & Kazman, R. (2012). *Software architecture in practice* (3rd ed.). Addison-Wesley.
- Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). *Site reliability engineering: How Google runs production systems*. O'Reilly Media.
- Chung, L., Nixon, B., Yu, E., & Mylopoulos, J. (2000). *Non-functional requirements in software engineering*. Springer.

- Fenton, N., & Bieman, J. (2014). *Software metrics: A rigorous and practical approach* (3rd ed.). CRC Press.
- Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The science of lean software and DevOps*. IT Revolution Press.
- International Organization for Standardization. (2011). *ISO/IEC 25010: Systems and software engineering—Systems and software quality requirements and evaluation (SQuaRE)—System and software quality models*. ISO.
- International Organization for Standardization. (2013). *ISO/IEC/IEEE 29119 software testing standard*. ISO.
- Kim, G., Humble, J., Debois, P., & Willis, J. (2016). *The DevOps handbook*. IT Revolution Press.
- Sommerville, I. (2011). *Software engineering* (9th ed.). Pearson.